Bache 0 — Contrato de API REST v1

Versión del documento: 1.1
Versión de la API: v1
Estado: decisiones funcionales y técnicas cerradas para implementar incrementalmente
Fecha: 20 de octubre de 2026
Equipo: Bastián Mansilla, David Ojeda y Elías Henríquez
Stack: React + Vite, Node.js + Express y PostgreSQL
Arquitectura: monolito modular por capas
Base local: `http://localhost:3000`
Prefijo: `/api/v1`

> Este es un contrato único para Bache 0 completo. Los hitos organizan la implementación, pero no generan contratos separados. Una ruta marcada para una etapa futura forma parte del diseño, pero no debe anunciarse como operativa hasta que exista código probado.

1. Archivos de referencia

- `docs/openapi.yaml`: especificación técnica de rutas, parámetros, solicitudes, respuestas y esquemas.
- `docs/contrato-api-rest-v1.md`: decisiones de diseño y reglas funcionales legibles por el equipo.
- `docs/contrato-api-rest-v1.pdf`: copia de lectura del Markdown.
- La tabla que se incluya en el informe técnico debe coincidir con `openapi.yaml`. Si se cambia un endpoint o campo, se actualizan los documentos en el mismo commit.

2. Decisiones canónicas del producto

1. Existe una sola API REST bajo `/api/v1` y las propiedades JSON utilizan `snake_case`.
2. Las fechas se transmiten en ISO 8601 UTC y PostgreSQL utiliza `TIMESTAMPTZ`.
3. El ciudadano no crea cuentas, no inicia sesión y no consulta un historial personal. El formulario es un flujo público de reporte como invitado.
4. Solo los funcionarios municipales y los superadministradores tienen cuentas en `usuarios`. El login municipal se implementará al final del Hito 2.
5. La identidad no se deduce solo del dominio del correo. Una cuenta municipal debe haber sido creada por un actor autorizado o por el proceso inicial de configuración.
6. El rol y la municipalidad autorizada se obtienen del registro vigente del usuario en la base de datos después de validar el JWT. No se confía en un `rol` o `municipalidad_id` enviado por React.
7. En el flujo ciudadano, el dispositivo obtiene las coordenadas; la interfaz muestra un pin que la persona confirma o corrige. El cliente no envía un nombre de comuna ni elige libremente la municipalidad de destino.
8. La API vuelve a validar la ubicación en el backend al crear el reporte. Dirección, calle, comuna, región y municipalidad asignada son datos derivados, no campos autoritativos enviados por el ciudadano.
9. Si la ubicación resuelve a una comuna sin cobertura activa de Bache 0, el reporte no se crea. Se devuelve un error `422 MUNICIPALIDAD_SIN_COBERTURA` con el mensaje: `La comuna de {comuna_detectada} todavía no cuenta con el servicio Bache 0. Si la ubicación no es correcta, mueve el pin y vuelve a verificarla.`

10. Si la comuna no puede identificarse de forma fiable, no se inventa una asignación ni se crea el reporte. Se solicita ajustar el pin e intentar nuevamente.
11. La calle puede ser desconocida cuando sí se identifica de forma fiable una comuna con cobertura. En ese caso se permite el reporte, con calle/dirección nulas y estado de geocodificación `parcial`.
12. La clasificación vial (expresa, troncal, colectora, de servicio o local) solo se informa si se obtiene de una fuente vial adecuada. Si no existe una fuente fiable, se deja `null` con estado `pendiente` o `no_disponible`; no se infiere por el nombre de la calle.
13. Las fotos se transmiten como archivo `multipart/form-data`. PostgreSQL conserva metadatos y una referencia, no el binario de la imagen.
14. Reportes y usuarios se archivan/desactivan lógicamente; no se borran físicamente a través del CRUD normal.
15. Las entidades modificables tienen `created_at` y `updated_at`. PostgreSQL actualiza `updated_at` mediante trigger. Además, las mutaciones relevantes crean eventos de auditoría independientes que no se sobrescriben.
16. La visión artificial se integra después mediante `VisionService`, prevista para Hito 3. Ni el proveedor ni el modelo quedan fijados en este contrato, y la IA no decide por sí sola la severidad o prioridad final.

3. Arquitectura y responsabilidades de las capas



- **Routes:** definen rutas y verbos HTTP.
- **Controllers:** adaptan los objetos HTTP y seleccionan códigos de respuesta; no ejecutan SQL.
- **Services:** validan reglas de negocio, autorización, cobertura, coordinación de almacenamiento e integración externa.
- **Repositories:** ejecutan consultas parametrizadas y operaciones de persistencia PostgreSQL.
- **Adaptadores:** implementan almacenamiento de fotos y proveedor de geocodificación detrás de una interfaz; la clave del proveedor nunca vive en React.
- **Middlewares:** autenticación JWT, autorización, validación compartida, límite de solicitudes y errores centralizados.

No se permite SQL dentro de controllers ni reglas de negocio críticas dentro de React.

4. Autenticación y autorización municipal

4.1 Cuenta y login

- Solo existe login para funcionarios municipales y superadministradores: `POST /api/v1/auth/login`.
- No existen rutas de registro, login, perfil ni historial para ciudadanos.
- El correo se normaliza a minúsculas y debe ser único sin distinguir mayúsculas/minúsculas.
- La contraseña debe tener al menos 12 caracteres y un máximo de 128. El servidor guarda solo un hash Argon2id; nunca la contraseña original.
- Un login exitoso devuelve un JWT de acceso tipo Bearer con vigencia de **30 minutos**. No habrá refresh token en esta versión; al expirar, el funcionario debe iniciar sesión nuevamente.
- El JWT incluye `sub`, `iss`, `aud`, `iat`, `exp` y un identificador de token. Las decisiones de rol, estado de cuenta y municipalidad se consultan contra el registro vigente de la base de datos.
- La clave `JWT_SECRET` se genera aleatoriamente, tiene al menos 32 bytes de entropía y solo existe en entorno del servidor. Nunca se sube a GitHub ni se envía al frontend como configuración.
- Login: máximo 5 intentos fallidos por combinación de origen y cuenta en 15 minutos. Se responde `429` al exceder el límite y `401` con mensaje genérico para credenciales incorrectas, sin revelar si el correo existe.
- El endpoint de login no registra contraseñas ni tokens en logs.
- El servidor vuelve a comprobar que la cuenta esté activa y carga su rol y municipalidad en cada petición protegida. Desactivar una cuenta impide usarla, aunque su JWT no haya expirado.
- `GET /api/v1/auth/me` devuelve el perfil municipal del token actual, sin hash ni secretos.
- `POST /api/v1/auth/change-password` permite cambiar la contraseña propia, exigiendo la contraseña actual. No se implementa recuperación pública por correo en H2.
- El primer superadministrador se crea mediante un proceso local de inicialización controlado; nunca mediante registro público ni con una contraseña predeterminada escrita en el repositorio.

4.2 Roles

- `inspector`: puede consultar reportes y fotografías de su municipalidad y cambiar el estado operativo de reportes de ese ámbito.
- `administrador_municipal`: puede consultar y gestionar reportes de su municipalidad y administrar cuentas de inspector de esa misma municipalidad. No puede crear superadministradores, cambiarse a otra municipalidad ni conceder permisos globales.
- `superadministrador`: puede consultar reportes de todas las municipalidades, crear/desactivar municipalidades, gestionar administradores municipales y resolver asignaciones cuando una fuente geográfica válida o una corrección autorizada lo requiera.

El `municipalidad_id` de un funcionario municipal no puede ser nulo. Solo `superadministrador` puede tener `municipalidad_id = null`.

4.3 Alcance de datos

- Un usuario municipal solo consulta/actualiza reportes cuyo `municipalidad_id` coincida con su cuenta.
- La municipalidad se obtiene desde la cuenta autenticada, no desde un filtro libre del cliente. Si un usuario municipal intenta filtrar otra municipalidad, se devuelve `403`.
- Si un usuario municipal solicita un ID que pertenece a otra municipalidad, se devuelve `404` para no revelar la existencia del recurso.
- Crear un usuario municipal está protegido. Un administrador municipal solo puede crear inspectores de su propia municipalidad. Solo superadministración crea administradores municipales o

superadministradores.
- Estos controles deben funcionar en backend y repositorio, no solo ocultando botones en React.

4.4 Pruebas locales

El flujo previsto en la colección de pruebas comienza creando/sembrando una cuenta de desarrollo controlada, llama al endpoint de login y usa el JWT devuelto en las solicitudes protegidas. No se permite un bypass de autenticación en un servidor accesible desde Internet. Antes de implementar el login, las rutas administrativas se prueban solo en una instancia local ligada a `localhost`; para la entrega, el login real queda implementado al final del Hito 2.

5. Reglas de fotografía

- Una fotografía por reporte en la primera versión.
- Formatos aceptados: JPEG, PNG y WebP.
- Tamaño máximo del archivo recibido: **10 MB**. Se rechazan archivos vacíos, corruptos o que no coincidan con su firma real, aunque el navegador declare un MIME permitido.
- SVG, GIF, PDF, vídeo y otros tipos quedan rechazados con `415 UNSUPPORTED_MEDIA_TYPE`.
- El backend valida el tipo y tamaño, normaliza la imagen con una biblioteca de procesamiento mantenida y elimina metadatos EXIF innecesarios al generar la copia de almacenamiento.
- El nombre original del usuario no se utiliza como ruta de almacenamiento. Se genera un identificador aleatorio y se evita sobrescribir archivos existentes.
- H2 usa almacenamiento local configurable fuera del directorio público del servidor, reproducible desde el README. La interfaz `StorageService` permite reemplazarlo luego por almacenamiento cloud sin cambiar el contrato.
- En PostgreSQL se guardan identificador/referencia, tipo de contenido, tamaño, dimensiones si están disponibles y fechas; no el binario completo.
- `foto_url` apunta a `GET /api/v1/reportes/{id}/foto`, no a una URL pública ni a una ruta física. La lectura exige permisos para consultar el reporte.
- Si el reporte falla por cobertura o validación antes de ser creado, cualquier archivo temporal debe eliminarse. No deben quedar archivos huérfanos.
- `POST /api/v1/reportes`: límite inicial de 5 intentos por IP cada 10 minutos; `POST /ubicaciones/resolver`: 20 solicitudes por IP por minuto. Límites configurables por entorno y respuesta `429` con `Retry-After`.

6. GPS, pin y cobertura de servicio

6.1 Flujo ciudadano

1. El navegador solicita permiso de geolocalización y propone un punto GPS.
2. El mapa muestra un marcador que el ciudadano puede confirmar o mover hasta la ubicación del desperfecto. La ubicación del teléfono no se da por idéntica a la del bache: se permite mover el pin.
3. Si se usa GPS con precisión estimada peor que 50 metros, la interfaz pide corregir el punto. Al colocar el pin manualmente, `metodo_ubicacion` pasa a `pin_manual` y `precision_ubicacion_m` puede ser `null` porque la precisión GPS ya no describe el punto elegido.
4. Si el permiso de GPS se deniega, se permite seleccionar el punto manualmente en el mapa. Si la persona no consigue aportar ni GPS ni un pin válido, el envío queda bloqueado con una explicación clara.

5. El frontend llama `POST /api/v1/ubicaciones/resolver` para mostrar dirección, comuna y cobertura antes de enviar. El backend aplica los mismos chequeos al crear el reporte; no confía en la respuesta previa del cliente.

6.2 Reglas de resolución

- La resolución geográfica la ejecuta `LocationService` mediante un adaptador configurado en servidor. La elección técnica del proveedor puede cambiar sin cambiar la API pública.
- Si se resuelve comuna y región, el backend las compara con la lista de municipalidades activas que tienen el servicio habilitado.
- Si la comuna no está habilitada, `POST /ubicaciones/resolver` y `POST /reportes` responden `422 MUNICIPALIDAD_SIN_COBERTURA`, incluyendo la comuna detectada y el mensaje: `La comuna de {comuna_detectada} todavía no cuenta con el servicio Bache 0. Si la ubicación no es correcta, mueve el pin y vuelve a verificarla.` No se crea ni se persiste el reporte.
- Si no puede identificarse comuna/región con fiabilidad, responde `422 UBICACION_NO_RESUELTA` con el mensaje: `No pudimos confirmar la comuna de esta ubicación. Mueve el pin a la ubicación del bache y vuelve a intentarlo.` No se crea el reporte.
- Si la comuna está cubierta pero no se encuentra una calle/dirección precisa, el reporte puede continuar después de que el usuario confirme el pin. Calle y dirección quedan nulas; el backend no inventa un valor.
- Si el proveedor de geocodificación está temporalmente caído, responde `503 UBICACION_TEMPORALMENTE_NO_DISPONIBLE`. El frontend permite reintentar y el reporte no se crea hasta que pueda validarse la cobertura.
- En caso de ambigüedad cerca de un límite comunal, se pide ajustar el pin y volver a resolver. Nunca se asigna “la comuna más cercana” como sustituto de una resolución no fiable.
- El backend establece `municipalidad_id` usando la resolución y la cobertura configurada; el cliente no puede enviarlo ni forzarlo.
- La clasificación vial se obtiene solo de una fuente vial pertinente. No es requisito para enviar un reporte y permanece nula con estado `pendiente`/`no_disponible` cuando no hay información fiable.

6.3 Campos de ubicación en `POST /reportes`

`multipart/form-data` incluye `foto`, `latitud`, `longitud`, `metodo_ubicacion` (`gps` o `pin_manual`), `ubicacion_confirmada=true`, `severidad`, `concurrencia_vial`, `infraestructura_critica` y `descripcion` opcional. `precision_ubicacion_m` es opcional y nula cuando el punto se elige manualmente.

El cliente **no envía** `direccion_detectada`, `calle_detectada`, `comuna_detectada`, `region_detectada`, `municipalidad_id`, `estado`, `folio`, `foto_url`, `clasificacion_vial` ni datos de IA. El backend los genera o valida.

7. Recursos y endpoints

Todos los endpoints están definidos con esquemas completos en `openapi.yaml`. Esta tabla resume el contrato humano y su etapa prevista.

7.1 Salud y municipalidades

Método y ruta	Acceso	Etapa	Regla principal
`GET /health`	Público	H2	Comprueba que el proceso API responda; no expone configuración ni secretos.
`GET /api/v1/municipalidades`	Público para municipalidades activas con servicio habilitado; superadmin puede incluir las inactivas	H2	Lista el catálogo de cobertura para la resolución geográfica. No permite al ciudadano elegir libremente una municipalidad de destino.
`GET /api/v1/municipalidades/{id}`	Público si está activa	H2	Devuelve información general de la municipalidad activa.
`POST /api/v1/municipalidades`	Superadmin	H3/evolución	Crea una municipalidad y define si el servicio está habilitado.
`PUT /api/v1/municipalidades/{id}`	Superadmin	H3/evolución	Actualiza el registro; se conserva historia.
`DELETE /api/v1/municipalidades/{id}`	Superadmin	H3/evolución	Desactiva lógicamente, no elimina físicamente.

7.2 Ubicación y autenticación municipal

Método y ruta	Acceso	Etapa	Regla principal
`POST /api/v1/ubicaciones/resolver`	Público, con límite de solicitudes	H2	Devuelve resolución completa/parcial o error explícito de ubicación/cobertura. No crea reporte.
`POST /api/v1/auth/login`	Público, con límite de intentos	H2, última fase	Solo funcionarios municipales/superadmin; devuelve JWT de 30 minutos.
`GET /api/v1/auth/me`	JWT válido	H2, última fase	Devuelve la identidad y rol vigentes del funcionario.
`POST /api/v1/auth/change-password`	JWT válido	H2, última fase	Exige contraseña actual y actualiza el hash.

No existen `POST /auth/registro`, `GET /usuarios/me/reportes` ni endpoints de login/registro ciudadano.

7.3 Reportes

Método y ruta	Acceso	Etapa	Regla principal
`POST /api/v1/reportes`	Público como invitado, con limitación de solicitudes	H2	Recibe foto y contexto, resuelve/coteja cobertura en backend y persiste solo si la comuna está cubierta y el pin está confirmado.
`GET /api/v1/reportes`	Inspector/admin municipal/superadmin	H2	Listado paginado; el ámbito municipal procede del token validado.
`GET /api/v1/reportes/{id}`	Inspector/admin municipal/superadmin	H2	No revela reportes de otra municipalidad a usuarios municipales.
`GET /api/v1/reportes/{id}/foto`	Igual que leer el reporte	H2	Devuelve la fotografía sin exponer rutas físicas.
`PUT /api/v1/reportes/{id}`	Admin municipal de su municipio/superadmin	H2	Actualiza los campos de gestión permitidos y genera auditoría.
`PATCH /api/v1/reportes/{id}/estado`	Inspector/admin municipal de su municipio/superadmin	H2	Cambia estado operativo a `pendiente`, `en_revision` o `reparado`.
`DELETE /api/v1/reportes/{id}`	Admin municipal de su municipio/superadmin	H2	Archivo lógico (`archivado_at`); respuesta `204`, sin borrar historia.
`PUT /api/v1/reportes/{id}/ubicacion`	Admin municipal de su municipio/superadmin	H3	Corrige coordenadas/metadatos y vuelve a validar cobertura; no permite cambiar libremente la municipalidad.
`PUT /api/v1/reportes/{id}/asignacion`	Superadmin	H3/evolución	Reasigna solo con motivo justificado y evento de auditoría.

Método y ruta	Acceso	Etapa	Regla principal
`GET /api/v1/reportes/{id}/historial`	Admin municipal del ámbito/superadmin	H3/evolución	Consulta eventos append-only. Los eventos empiezan a registrarse durante H2.

7.4 Usuarios municipales

Método y ruta	Acceso	Etapa	Regla principal
`GET /api/v1/usuarios`	Admin municipal del ámbito/superadmin	H2	Lista usuarios sin hashes ni datos de autenticación.
`POST /api/v1/usuarios`	Admin municipal/superadmin, con restricción de rol y municipio	H2	Recibe contraseña inicial; Service genera Argon2id. El admin municipal solo crea inspectores propios.
`GET /api/v1/usuarios/{id}`	Admin municipal del ámbito/superadmin	H2	No devuelve el hash.
`PUT /api/v1/usuarios/{id}`	Admin municipal del ámbito/superadmin	H2	Actualiza datos permitidos; admin municipal no puede cambiar la cuenta a otro municipio ni elevar privilegios.
`DELETE /api/v1/usuarios/{id}`	Admin municipal del ámbito/superadmin	H2	Desactiva cuenta y conserva historia.

7.5 Visión artificial

Método y ruta	Acceso	Etapa	Regla principal
`POST /api/v1/reportes/{id}/analisis-vision`	Usuarios municipales autorizados	H3	Solicita análisis asíncrono; devuelve `202 Accepted`.
`GET /api/v1/reportes/{id}/analisis-vision`	Usuarios municipales autorizados	H3	Lista ejecuciones de visión asociadas al reporte.
`GET /api/v1/analisis-vision/{id}`	Usuarios municipales autorizados	H3	Devuelve estado, modelo/versión, detecciones y errores controlados.

8. Convención de respuesta y errores

Respuesta individual

```
{
  "data": {
    "id": 123,
    "folio": "BA-2026-00123"
  }
}
```

Listado paginado

```
{
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 0,
    "total_pages": 0
  }
}
```

Error uniforme

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Hay campos que requieren corrección.",
    "details": [
      { "field": "severidad", "issue": "Debe ser baja, media o alta." }
    ],
    "request_id": "req_example123"
  }
}
```

`details` siempre es una lista, incluso vacía. `request_id` permite correlacionar el error con el registro interno. Nunca se exponen stack traces, SQL, rutas físicas, contraseñas, hashes, JWT ni claves de proveedores.

HTTP	Código habitual	Regla
200	—	Consulta, resolución de ubicación o actualización exitosa.
201	—	Recurso creado y persistido.
202	—	Solicitud de análisis aceptada para procesamiento futuro.

HTTP	Código habitual	Regla
204	—	Archivo lógico/desactivación realizada; cuerpo vacío.
400	VALIDATION_ERROR	JSON, query, ID o campo mal formado/invalidado.
401	AUTHENTICATION_REQUIRED / INVALID_CREDENTIALS	Token ausente/inválido/expirado o credenciales incorrectas. Mensaje de login genérico.
403	FORBIDDEN	Usuario autenticado sin permiso.
404	NOT_FOUND	Recurso inexistente o fuera de ámbito del usuario municipal.
409	CONFLICT	Email duplicado u operación incompatible con el estado actual.
413	FILE_TOO_LARGE	Foto supera 10 MB.
415	UNSUPPORTED_MEDIA_TYPE	Archivo fuera de JPEG/PNG/WebP o firma de archivo no válida.
422	MUNICIPALIDAD_SIN_COBERTURA, UBICACION_NO_RESUELTA, LOCATION_PRECISION_LOW	La petición está bien formada, pero no cumple reglas de ubicación/cobertura. No se crea reporte.
429	RATE_LIMIT_EXCEEDED	Exceso de solicitudes; incluir Retry-After cuando sea aplicable.
500	INTERNAL_SERVER_ERROR	Falla inesperada. Mensaje genérico al cliente; detalle solo en log interno.
503	UBICACION_TEMPORALMENTE_NO_DISPONIBLE / SERVICE_UNAVAILABLE	Dependencia indispensable no disponible temporalmente. No se persiste un reporte incompleto.

No se responde 200 ante errores. Se mantienen consistentes los códigos HTTP y el cuerpo uniforme de error.

9. Reglas de los datos y PostgreSQL

9.1 Tablas objetivo

Tabla	Propósito y reglas
municipalidades	Catálogo de entidades/comunas cubiertas. Incluye nombre, comuna, región, activa, servicio_activo, created_at, updated_at. La combinación normalizada comuna+región no se duplica.

Tabla	Propósito y reglas
`usuarios`	Solo funcionarios y superadministradores. Incluye nombre, email normalizado único, `password_hash`, rol, `municipalidad_id`, `activo`, fechas. `municipalidad_id` no nulo para roles municipales y nulo solo para superadmin.
`reportes`	Folio, coordenadas, precisión, método de ubicación, confirmación, metadatos geográficos, `municipalidad_id`, severidad, estado, concurrencia, infraestructura crítica, descripción, referencia/metadata de foto, fechas y `archivado_at`. No existe `usuario_id` ciudadano.
`analisis_vision`	Una fila por ejecución de análisis futura, relacionada con reporte; conserva proveedor, modelo/versión, estado, detecciones JSONB/error y fechas.
`auditoria_eventos`	Registro append-only de actor, tipo de actor (`funcionario`, `invitado` o `sistema`), entidad, ID, acción, cambios autorizados antes/después y `created_at`. No se actualiza ni se borra por la API normal.

9.2 Claves, restricciones e índices

- PK entera autogenerada para entidades principales; folio público no sustituye el ID interno.
- FK: usuario municipal -> municipalidad; reporte -> municipalidad; análisis -> reporte; evento -> actor cuando exista una cuenta.
- La cuenta pública del ciudadano no se persiste: para reportes invitados no se guarda un usuario creador.
- Emails normalizados en minúscula y únicos; hash obligatorio para usuarios municipales; rol limitado a `inspector`, `administrador_municipal`, `superadministrador`.
- Coordenadas dentro de latitud `[-90,90]` y longitud `[-180,180]`; severidad `baja / media / alta`; estado `pendiente / en_revision / reparado`; concurrencia `baja / media / alta / muy_alta`; infraestructura `ninguna / colegio / centro_salud / estacion_terminal / otra`.
- Foto: referencia única por reporte en H2, tipo MIME permitido, tamaño en bytes y nombre interno aleatorio.
- Índices recomendados: email único normalizado; reportes por `(municipalidad_id, estado, created_at)`; reportes por `created_at`; auditoría por `(entidad, entidad_id, created_at)`; análisis por `(reporte_id, created_at)`.
- `ON DELETE CASCADE` no se utilizará para eliminar historial de reportes. Las relaciones se preservan con archivo/desactivación lógica.

9.3 Fechas y auditoría

- Todas las tablas de negocio mutables (`municipalidades`, `usuarios`, `reportes`, `analisis_vision`) tienen `created_at` y `updated_at TIMESTAMPTZ NOT NULL`.
- `created_at` se asigna al crear y no cambia. Un trigger PostgreSQL actualiza `updated_at` en cada `UPDATE`; `DEFAULT NOW()` por sí solo no basta.
- `reportes.archivado_at` se establece al archivar; `usuarios.activo=false` al desactivar una cuenta/municipalidad.
- `auditoria_eventos` tiene `created_at`, pero no `updated_at`: un evento representa un hecho inmutable. La API ordinaria no puede editar ni borrar eventos.

- En H2 se registra creación de reportes, cambios de reportes/estado, archivo de reportes, altas/cambios/desactivaciones de usuarios y cambios de configuración municipal. El actor puede ser nulo para creación ciudadana invitada; se usa `tipo_actor=invitado`.
- Guardar solo los cambios relevantes, sin contraseñas, hashes, tokens, secretos ni imágenes completas en el JSON de auditoría. La consulta del historial se habilita en H3, pero el registro comienza en H2.

10. Validación, consistencia e idempotencia

- Todas las validaciones críticas se repiten en backend aunque el frontend valide.
- IDs de path son enteros positivos; IDs inválidos generan `400`.
- Paginación: `page >= 1`, `limit` por defecto 20, máximo 100.
- Descripción opcional, máximo 2.000 caracteres; texto se normaliza sin aceptar contenido ejecutable como HTML.
- Solo se aceptan enumeraciones definidas en este contrato; las etiquetas visibles para personas se traducen en el frontend.
- Un `POST /reportes` no queda persistido hasta que archivo, datos, ubicación y cobertura hayan sido validados. La inserción del reporte y sus metadatos debe ser transaccional; si falla, se limpia archivo temporal.
- Los errores de la dependencia geográfica no se interpretan como “sin cobertura”; son errores distintos. No declarar una comuna fuera de cobertura si el proveedor está caído.
- El backend no debe guardar reportes duplicados por doble clic: el frontend desactiva temporalmente el botón durante el envío y el backend puede aceptar `Idempotency-Key` en `POST /reportes` para proteger reintentos. Si se soporta, la misma clave con el mismo contenido devuelve el resultado previo; con contenido distinto devuelve `409`.
- No se devuelve el identificador de un recurso de otra municipalidad a un actor municipal.

11. Visión artificial (Hito 3)

El backend invocará un `VisionService` desacoplado del proveedor. La solicitud `POST /reportes/{id}/analisis-vision` devuelve `202 Accepted` con el ID y estado `pendiente`; la consulta recupera `pendiente`, `procesando`, `completado` o `fallido`, proveedor, modelo, versión, detecciones, `analizado_at` y código de error controlado.

Una detección puede incluir etiqueta, confianza entre 0 y 1 y caja delimitadora normalizada. No representa automáticamente profundidad, severidad normativa ni prioridad de reparación. El modelo y proveedor se seleccionan después de pruebas comparativas y revisión de licencias, costo, límites, tratamiento de fotos y rendimiento. El ciudadano no espera la inferencia para enviar el reporte.

12. Etapas de implementación

Hito 2 — núcleo operativo

- Servicio Express ejecutable y `GET /health`.
- PostgreSQL reproducible mediante `db/schema.sql` y `db/seed.sql`.
- CRUD real de reportes y usuarios con Routes -> Controllers -> Services -> Repositories.
- Reporte ciudadano público sin cuenta, con foto, coordenadas/pin, validación de cobertura y contexto.
- Login municipal JWT y protección real de endpoints administrativos, implementados al final del hito.
- Reglas de acceso por rol y aislamiento por `municipalidad_id` para las rutas implementadas.

- Fotos en almacenamiento local protegido detrás de `StorageService`; `.env`, `.env.example` y `.gitignore` correctos.
- Validación y error uniforme, timestamps automáticos y creación de eventos de auditoría en mutaciones relevantes.
- Colección importable de API y documento técnico con la tabla que coincide con OpenAPI.

Hito 3 / evolución

- Clasificación vial con fuente geográfica adecuada y mejoras de corrección de ubicación.
- Consulta del historial de auditoría, controles operacionales adicionales y almacenamiento cloud.
- Visión artificial mediante `VisionService` después de evaluar un modelo/proveedor.

El estado de etapa es un plan de implementación, no permiso para fingir que una ruta futura ya funciona. Si algo queda pendiente, se documenta como tal.

13. Criterios para declarar el contrato aceptado

- `openapi.yaml` valida como OpenAPI 3.1 y no contiene rutas de cuentas ciudadanas.
- Cada endpoint declara acceso, entrada, respuesta de éxito y errores relevantes.
- Los mismos nombres y enumeraciones aparecen en OpenAPI, el código, SQL y frontend.
- Un ciudadano puede enviar un reporte sin cuenta.
- Una comuna sin cobertura produce el mensaje definido y no crea un reporte.
- Un funcionario municipal no puede consultar/alterar reportes de otra municipalidad.
- Las fotos se validan, se almacenan fuera del directorio público y se sirven con autorización.
- `created_at`/`updated_at` son consistentes y cada mutación relevante crea evento de auditoría.
- Las rutas H3 están marcadas como futuras y no se presentan como implementadas.
- La tabla del informe técnico coincide con OpenAPI.

Este documento define el contrato; no certifica que los endpoints estén implementados. La aceptación operativa requiere ejecutar la aplicación, levantar PostgreSQL desde cero y pasar las pruebas CRUD y de errores en el entorno local.